

Implementación en la VPS

Marca de agua · tamaño de píxeles en tiempo real

Todo lo que falta en el backend (server.js) para dejarlo listo de una vez.

Los DOS reproductores ya están compilados e implementados: el de PC (Electron/Windows) y el APK (Android). Ambos abren un canal SSE y re-renderizan el tamaño/color/grosor de la marca de agua en vivo, sin recargar el video. Este documento cubre exclusivamente lo que falta implementar tú en la VPS para que el sistema funcione de extremo a extremo.

Estado actual

- Reproductor PC: implementado en player-app/renderer/player.js (subscribeWmConfig / EventSource / re-render).
- Reproductor APK: implementado en PlayerActivity.kt (subscribeWmConfig con OkHttp, os=android).
- VPS (este documento): faltan las funciones de config, el canal SSE, el broadcast, las rutas admin y añadir watermarkConfig + courseld a las respuestas de reproducción.
- Infra: VPS propia (<https://reproductorsatanafxpro.dpdns.org>). NO se usa Render.

0 · Contrato que ya consumen los reproductores

El backend debe respetar exactamente este contrato (los reproductores ya están escritos así):

Qué	Detalle exacto
Canal SSE	GET {base}/api/watermark/stream/:courseld?token={JWT}&os={windows android}
Evento SSE	Cada push es event: config + data: {JSON de la config}
Auth SSE	Por query ?token=JWT (EventSource/So no envían cabeceras). 401 si falla.
os enviado	PC envía os=windows · APK envía os=android
Config inicial	Campo watermarkConfig (objeto por SO) + courseld en la respuesta de reproducción
Data del evento	El reproductor toma data[os] data !' debes enviar el objeto multi-SO completo
Tamaño	size en píxeles, rango 6–80 (clamp en el servidor, nunca confíes en el cliente)

IMPORTANTE: el evento SSE debe llamarse exactamente "config" y el cuerpo ir como event: config\data: {...}\n\n . El reproductor escucha addEventListener("config", ...). Si cambias el nombre del evento, no se aplica nada.

1 · Modelo de datos + validación (clamp 6–80)

Estructura de la config: organizada por Sistema Operativo y, dentro, por marca (email, ip, code, datetime). Cada marca tiene: on, size (6–80), color (#rrggbb), weight (100–900), y un enabled global por SO.

Campos

Campo	Tipo	Rango	Significado
enabled	bool	true/false	Activa las marcas para ese SO
on	bool	true/false	Marca individual visible o no

size	int	6 – 80	Tamaño en píxeles (font-size)
color	string	#rrggbb	Color del texto
weight	int	100 – 900	Grosor de la fuente

1.1 · Defaults + saneo (pieza de seguridad clave)

Ajuste vs la guía original: el APK que compilamos usa el correo a 35 px (no 22). Por eso el default de android.email.size aquí es 35, para que no haya salto visual antes de que el admin edite. El resto de marcas en móvil = 11.

```

const WM_OS_LIST = ['web', 'windows', 'mac', 'linux', 'android', 'ios'];
const WM_MARKS = ['email', 'ip', 'code', 'datetime'];

// Valores por defecto por SO (nada cambia hasta que el admin edite)
function _wmDefaultMark(os, key) {
  const isDesktop = (os === 'windows' || os === 'mac' || os === 'linux');
  const isMobile = (os === 'android' || os === 'ios');
  if (key === 'email') {
    if (isDesktop) return { on:true, size:35, color:'#ff2d2d', weight:700 };
    if (isMobile) return { on:true, size:35, color:'#ff2d2d', weight:700 }; // APK=35
    return { on:true, size:16, color:'#ffffff', weight:600 }; // web
  }
  if (isMobile) return { on:true, size:11, color:'#ffffff', weight:600 };
  return { on:true, size:16, color:'#ffffff', weight:600 };
}
function buildDefaultWmOs(os) {
  const o = { enabled: true };
  WM_MARKS.forEach(k => { o[k] = _wmDefaultMark(os, k); });
  return o;
}
function buildDefaultWmConfig() {
  const c = {};
  WM_OS_LIST.forEach(os => { c[os] = buildDefaultWmOs(os); });
  return c;
}

// %% Saneo: aquí se fuerza el rango del TAMAÑO EN PÍXELES (6-80) %%
function _wmClampInt(v, min, max, def) {
  const n = parseInt(v, 10);
  if (!Number.isFinite(n)) return def;
  return Math.min(max, Math.max(min, n));
}
function _wmSanColor(v, def) {
  return (typeof v === 'string' && /^[0-9a-fA-F]{6}$/.test(v)) ? v.toLowerCase() : def;
}
function _wmSanMark(input, os, key) {
  const d = _wmDefaultMark(os, key);
  input = input || {};
  return {
    on: input.on === undefined ? d.on : !!input.on,
    size: _wmClampInt(input.size, 6, 80, d.size), // ª TAMAÑO px 6-80
    color: _wmSanColor(input.color, d.color),
    weight: _wmClampInt(input.weight, 100, 900, d.weight),
  };
}
function _wmSanOs(input, os) {
  const d = buildDefaultWmOs(os);
  input = input || {};
  const o = { enabled: input.enabled === undefined ? d.enabled : !!input.enabled };
  WM_MARKS.forEach(k => { o[k] = _wmSanMark(input[k], os, k); });
  return o;
}
function sanitizeWmConfig(input) {
  const out = {};
  WM_OS_LIST.forEach(os => { out[os] = _wmSanOs(input && input[os], os); });
  return out;
}

```

2 · Persistencia + resolución por curso

Guarda la config en tu store bajo la clave "watermark_configs". Si tu database.js / database-pg.js no tiene getConfig/setConfig, crea una tabla config(key TEXT PRIMARY KEY, value TEXT) o guarda un JSON en disco. La resolución mezcla con defaults y hereda de __default__ cuando el curso no tiene config propia.

```

async function loadWmConfigs() {
  try {
    const raw = await db.getConfig('watermark_configs');
    if (!raw) return {};
    const obj = JSON.parse(raw);
    return (obj && typeof obj === 'object') ? obj : {};
  } catch { return {}; }
}
async function saveWmConfigs(obj) {
  await db.setConfig('watermark_configs', JSON.stringify(obj || {}));
}

// Config de un curso, mezclada con defaults. Sin config propia !' __default__
function resolveWmConfig(configs, courseId) {
  const key = courseId || '__default__';
  const stored = (configs && configs[key])
    || (configs && configs['__default__']) || null;
  if (!stored) return buildDefaultWmConfig();
  return sanitizeWmConfig(stored);
}

```

3 · Registro de suscriptores SSE + broadcast

Un Map en memoria mantiene, por curso, el conjunto de conexiones SSE abiertas. Cuando el admin guarda, se empuja la config nueva a todos los reproductores de ese curso.

```

// courseKey -> Set<res> (conexiones SSE abiertas por curso)
const _wmStreams = new Map();
function _wmSubscribe(courseKey, res) {
  if (!_wmStreams.has(courseKey)) _wmStreams.set(courseKey, new Set());
  _wmStreams.get(courseKey).add(res);
}
function _wmUnsubscribe(courseKey, res) {
  const set = _wmStreams.get(courseKey);
  if (set) { set.delete(res); if (set.size === 0) _wmStreams.delete(courseKey); }
}
// Empuja la config nueva a los reproductores conectados de ese curso
async function broadcastWmConfig(courseKey) {
  const set = _wmStreams.get(courseKey);
  if (!set || set.size === 0) return;
  const configs = await loadWmConfigs();
  const cfg = resolveWmConfig(configs, courseKey);
  const data = JSON.stringify(cfg);
  for (const res of set) {
    try { res.write(`event: config\ndata: ${data}\n\n`); } catch {}
  }
}

```

El reproductor lee data[os] || data. Envía SIEMPRE el objeto multi-SO completo (resolveWmConfig ya lo devuelve con web/windows/android/...). Así el PC toma .windows y el APK toma .android del mismo push.

4 · Rutas Express (SSE + polling + admin)

4.1 · Canal SSE en tiempo real

```
// EventSource no envía cabeceras !' auth por query token
app.get('/api/watermark/stream/:courseId', async (req, res) => {
  let payload = null;
  try { payload = jwt.verify(String(req.query.token || ''), JWT_SECRET); }
  catch { payload = null; }
  if (!payload) return res.status(401).end();

  const courseKey = req.params.courseId || '__default__';
  res.set({
    'Content-Type': 'text/event-stream',
    'Cache-Control': 'no-cache, no-transform',
    'Connection': 'keep-alive',
    'X-Accel-Buffering': 'no', // desactiva buffering de nginx
  });
  if (typeof res.flushHeaders === 'function') res.flushHeaders();

  // 1) Config inicial inmediata
  const configs = await loadWmConfigs();
  const cfg = resolveWmConfig(configs, courseKey);
  res.write(`event: config\ndata: ${JSON.stringify(cfg)}\n\n`);

  // 2) Suscribirse + latido cada 25s (mantiene viva la conexión)
  _wmSubscribe(courseKey, res);
  const hb = setInterval(() => { try { res.write(': ping\n\n'); } catch {} }, 25000);
  req.on('close', () => { clearInterval(hb); _wmUnsubscribe(courseKey, res); });
});
```

4.2 · Polling puntual (opcional, fallback sin SSE)

```
app.get('/api/watermark/config/:courseId', requireAuth, async (req, res) => {
  const configs = await loadWmConfigs();
  res.json(resolveWmConfig(configs, req.params.courseId || '__default__'));
});
```

4.3 · Admin: cargar el editor

```
app.get('/api/admin/watermark/configs', requireAdmin, async (req, res) => {
  const configs = await loadWmConfigs();
  let courses = [];
  try { courses = (await db.getAllCourses()).map(c => ({ id:c.id, name:c.name })); }
  catch {}
  res.json({ osList: WM_OS_LIST, marks: WM_MARKS,
    defaults: buildDefaultWmConfig(), courses, configs });
});
```

4.4 · Admin: guardar + broadcast en tiempo real

```
app.post('/api/admin/watermark/config/:courseId', requireAdmin, async (req, res) => {
  const courseKey = req.params.courseId || '__default__';
  const clean = sanitizeWmConfig((req.body && req.body.config) || {});
  const configs = await loadWmConfigs();
  configs[courseKey] = clean;
  await saveWmConfigs(configs);
  broadcastWmConfig(courseKey).catch(() => {}); // %Ã push a reproductores
  res.json({ ok: true, courseId: courseKey, config: clean });
});
```

5 · Añadir watermarkConfig + courseId a las respuestas de reproducción

Los reproductores leen la config inicial del cuerpo de la respuesta cuando abren un video. Debes añadir dos campos — watermarkConfig (objeto multi-SO resuelto) y courseId — a la respuesta JSON de CADA endpoint de reproducción que usan las apps:

- GET /api/video/:videoid/play !' lo usa el APK (catálogo) y equivalente en PC
- POST /api/playback/resolve !' enlaces cdp://play?t= / cmd=
- POST /api/playback/resolve-perm !' enlaces permanentes cdp://play?p=

En cada uno, justo antes de responder, resuelve la config del curso del video y añádela:

```
// Dentro del handler de reproducción, cuando ya conoces el courseId del video:
const _wmConfigs = await loadWmConfigs();
const watermarkConfig = resolveWmConfig(_wmConfigs, courseId); // objeto multi-SO

return res.json({
  // ...campos que ya devuelves (manifestUrl, mediaToken, watermarkText, otp, ...)
  courseId,                // %Ã requerido por el reproductor para el canal SSE
  watermarkConfig,          // %Ã config inicial (evita el parpadeo de defaults)
});
```

Los modelos del APK ya esperan estos campos: PlayUrlResponse y ResolveResponse tienen watermarkConfig (JsonElement) y courseId. Si no los envías, el APK usa el canal SSE igual (con courseId="__default__"). Enviarlos es lo correcto para que cada curso tenga su tamaño y no haya parpadeo inicial.

6 · Nginx en la VPS (para que el SSE fluya al instante)

El SSE necesita que el proxy NO haga buffering y no corte la conexión larga. En el bloque location que sirve / api/ (o uno específico para el stream) añade:

```
location /api/watermark/stream/ {
  proxy_pass http://127.0.0.1:3000; # tu upstream de Node
  proxy_http_version 1.1;
  proxy_set_header Connection '';
  proxy_buffering off;              # %Ã imprescindible para SSE
  proxy_cache off;
  proxy_read_timeout 3600s;         # %Ã no cortar la conexión larga
  chunked_transfer_encoding off;
  add_header X-Accel-Buffering no;
}
```

La cabecera X-Accel-Buffering: no que ya pone el handler también desactiva el buffering por-respuesta. Igual conviene poner proxy_buffering off en nginx.conf para el resto de la infraestructura.

7 · Panel admin — el slider del tamaño (admin.html)

El control que mueves para cambiar el tamaño en vivo es un input range 6–80 con auto-guardado (debounce 700 ms). Al guardar, el POST del punto 4.4 hace el broadcast y todos los reproductores del curso cambian al instante.

7.1 · Render del editor por marca

```
function wmRenderMarks() {
  const cfg = _wmEnsureCourse(_wmCourse);
  const osCfg = cfg[_wmOs];
  const box = document.getElementById('wm-marks');
  box.innerHTML = _wmData.marks.map(mk => {
    const m = osCfg[mk] || { on:true, size:16, color:'#ffffff', weight:600 };
    return `
    <div class="mark-card">
      <b>${_WM_MARK_LABEL[mk]}</b>
      <label><input type="checkbox" ${m.on?'checked':''}
        onchange="wmSetMark('${mk}','on',this.checked)"> Visible</label>
      <!-- & SLIDER DEL TAMAÑO EN PÍXELES & -->
      <label>Tamaño: <b id="wm-size-lbl-${mk}">${m.size}px</b>
        <input type="range" min="6" max="80" value="${m.size}"
          oninput="wmSetMark('${mk}','size',this.value)">
      </label>
      <label>Color: <input type="color" value="${m.color}"
        onchange="wmSetMark('${mk}','color',this.value)"></label>
      <label>Grosor:
        <select onchange="wmSetMark('${mk}','weight',this.value)">
          <option value="400">Normal</option>
          <option value="600">Semi-negrita</option>
          <option value="700">Negrita</option>
          <option value="900">Extra negrita</option>
        </select>
      </label>
    </div>`;
  }).join('');
  wmRenderPreview();
}
```

7.2 · Cambio + auto-guardado con debounce

```
function wmSetMark(mk, field, val) {
  const m = _wmEnsureCourse(_wmCourse)[_wmOs][mk];
  if (field === 'on') m.on = !!val;
  if (field === 'size') { m.size = parseInt(val,10) || 16;
    const l = document.getElementById('wm-size-lbl-'+mk);
    if (l) l.textContent = m.size + 'px'; }
  if (field === 'color') m.color = val;
  if (field === 'weight') m.weight = parseInt(val,10) || 600;
  wmRenderPreview();
  wmScheduleAutoSave(); // ª dispara guardado + broadcast
}

let _wmSaveTimer = null;
function wmScheduleAutoSave() {
  clearTimeout(_wmSaveTimer);
  const course = _wmCourse;
  _wmSaveTimer = setTimeout(() => _wmPersist(course), 700);
}
async function _wmPersist(course) {
  const cfg = _wmEnsureCourse(course);
  const r = await api('POST',
    '/api/admin/watermark/config/' + encodeURIComponent(course), { config: cfg });
  _wmData.configs[course] = r.config;
  // El backend ya hizo broadcast !' los reproductores ya cambiaron de tamaño
}
```

8 · Checklist de implementación en la VPS

#	Ítem	Detalle
1	Funciones de config	WM_OS_LIST, WM_MARKS, defaults, _wmClampInt, sanitizeWmConfig, resolveWmConfig

2	Persistencia	loadWmConfigs / saveWmConfigs (clave watermark_configs en tu store)
3	Registro SSE	_wmStreams Map + _wmSubscribe / _wmUnsubscribe / broadcastWmConfig
4	Ruta SSE	GET /api/watermark/stream/:courseId (auth por ?token, event: config, heartbeat 25s)
5	Rutas admin	GET /api/admin/watermark/configs · POST /api/admin/watermark/config/:courseId
6	Respuestas play	Añadir courseId + watermarkConfig a /video:id/play, /playback/resolve, /resolve-perm
7	Nginx	proxy_buffering off · proxy_read_timeout 3600s en la location del stream
8	Panel admin	Slider 6–80 + auto-guardado (admin.html)
9	Clamp servidor	Nunca confiar en el size del cliente: siempre _wmClampInt(size,6,80)

9 · Seguridad y buenas prácticas

Riesgo	Mitigación
Tamaño inválido / abuso	_wmClampInt(size, 6, 80) en el servidor. El valor del cliente nunca se usa sin validar.
Color inyectado (XSS)	Regex /^[0-9a-fA-F]{6}\$/; cualquier otra cosa cae al color por defecto.
SSE sin auth	Se verifica el JWT del query antes de abrir el canal (401 si falla).
Solo admin edita	Middleware requireAdmin en las rutas de guardado/lectura del editor.
Conexiones colgadas	Heartbeat : ping cada 25 s y limpieza en req.on("close").
Fallo de red del cliente	EventSource / OkHttp reintentan solos; la marca conserva el último tamaño.

La marca de agua DOM/overlay es disuasoria/forense, no una protección criptográfica. Para protección fuerte combínala con watermarking en el propio stream y FLAG_SECURE (el APK ya lo aplica).

10 · Resumen de endpoints a crear

Método	Ruta	Rol	Función
GET	/api/watermark/stream/:courseId	alumno	Canal SSE tiempo real (event: config)
GET	/api/watermark/config/:courseId	alumno	Config puntual (polling, opcional)
GET	/api/admin/watermark/configs	admin	Cargar el editor
POST	/api/admin/watermark/config/:courseId	admin	Guardar + broadcast

Con estos 4 endpoints + los campos courseId/watermarkConfig en las respuestas de reproducción + el ajuste de nginx, el sistema queda completo: el admin mueve el slider y el tamaño en píxeles cambia al instante en el reproductor de PC y en el APK, sin recargar.

Documento generado para la VPS de Satana FX Player · marca de agua con tamaño de píxeles en tiempo real. Los reproductores (PC y APK) ya consumen este contrato.